Tidal: An SLA-Contracted Batch Tier for LLM Serving Engines

Gurunath Lunkupali Venugopal

August 6, 2026

CONTENTS

Tidal: An SLA-Contracted Batch Tier for LLM Serving Engines

Gurunath Lunkupali Venugopal

Abstract

Commercial LLM providers sell two products from the same weights: online inference, priced for latency, and a batch API priced at half the online rate with a 24-hour completion window. Self-hosted deployments have no equivalent of the second product, even though a decode-dominated engine iteration uses a small fraction of its per-iteration token budget. Research systems that co-locate offline work behind online traffic (HyGen, ConServe) recover much of this idle capacity, but none of them exposes what makes the batch product sellable: a customer-facing completion contract. We present Tidal, which adds a *contracted* batch tier to vLLM: an OpenAI-wire-compatible `/v1/batches` front end with durable storage; per-job **laxity-driven escalation** (classical Least-Laxity-First, applied per batch job against an online tier) that leaves on-track jobs at the lowest priority band and escalates at-risk jobs early, continuously, and independently; a token-bucket progress floor; and a laxity-based **admission feasibility check** that rejects deadlines the system's own arithmetic cannot meet. We implement the same policy at two placements — an external gateway over an unmodified engine, and an in-engine work-conserving scheduler deployed as a plugin (not a fork) whose interference model calibrates itself from live step timings with no offline profiling — and compare them on identical traces, a design axis prior systems do not evaluate. On a deliberately modest CPU testbed we validate the mechanisms and find a placement *inversion*: the gateway holds online p99 request latency to 1.76× baseline where the in-engine scheduler reaches 8.25×, because the binding interference channel (resident-decode contention) sits below the reach of admission-time control — while the in-engine scheduler harvests 71% of the offline throughput ceiling versus the gateway's 45%, making placement a throughput/latency frontier rather than a ranking. We are explicit about what the testbed cannot show, including that no tested load regime discriminated the escalation machinery end-to-end. The implementation, including a client that plans coding work onto batch APIs, is open source.

1. Introduction

The economics of LLM serving are dominated by GPU hours, and GPU hours are dominated by waste. An engine serving interactive chat traffic runs its scheduler every few tens of milliseconds; in each iteration it may decode one token for each of a few dozen live requests while its compute budget — the number of tokens the hardware could process in that same iteration at acceptable latency — sits in the thousands. Decode is memory-bandwidth-bound: the weights stream from HBM whether the iteration carries 20 tokens or 2,000. The marginal cost of adding prefill work to a decode-heavy iteration is small until a compute knee, a fact exploited by chunked-prefill schedulers [sarathi] and by every co-location system since.

Providers monetize this slack directly: submit a file of requests, receive results within 24 hours, pay half the online price. A self-hosted deployment running vLLM cannot offer this product. vLLM ships an offline batch runner that requires dedicating the machine and an online server with no batch-job surface; a proposal to add SLA tiers to its scheduler was closed without adoption [vllm-rfc-30256].

The research literature has a different gap. HyGen [hygen] and ConServe [conserve] co-locate offline requests behind online traffic with strict priority and preemption, recovering 78–88% of dedicated-offline throughput while protecting online SLOs — and both include mechanisms that temper offline starvation. What neither exposes is a *customer-facing completion contract*: a specific job, a specific deadline, admission that refuses infeasible work, and scheduling that spends exactly as much online headroom as that deadline requires. Deadline-aware LLM schedulers exist — Niyama [niyama] co-schedules QoS classes with per-request deadlines and claims QoS targets — but at second-to-minute horizons, with deadlines as optimization targets and graceful degradation under overload. The batch-API semantics that OpenAI and Anthropic actually sell — day-scale windows, per-job, with an admission decision — appear in none of them.

The contract is the product. A customer submits an overnight batch because “done by morning” is dependable enough to plan around. Our motivating client, lazycode [lazycode] — a coding agent that plans work with a realtime model and executes it on provider batch APIs at the discount — exists only because that window is dependable. Making self-hosted batch serving sellable requires promoting the deadline from an optimization target to a contract with an admission face and a scheduling face.

Tidal implements both faces with deliberately classical machinery. Per batch job, the system tracks *laxity* — time remaining minus work remaining at the observed completion rate. At **admission**, a job whose projected completion exceeds its window (with margin) is refused outright — the self-hosted analogue of commercial APIs rejecting oversized jobs. In **scheduling**, urgency ramps only inside a short escalation horizon of remaining slack: an on-track job never rises above the lowest priority band, no matter how long it has been running (we document how easily this property is silently lost: our first implementation scaled urgency by the full window, reintroducing wall-clock aging — a healthy job at hour 12 escalated for no reason; an adversarial cross-model review caught it, and a regression test now pins the hour-12 case). An at-risk job escalates early, continuously, and independently of other jobs, avoiding synchronized floods; at maximum urgency it sits one notch below online priority with a token-bucket floor guaranteeing forward progress — the floor also closes the loop, since guaranteed progress raises laxity, stabilizing an escalated job at the floor rather than oscillating. A configuration flag (`sla_strict`) allows true priority parity at the deadline; we treat strictness as a policy dial and leave measuring its latency price to future work. Least-Laxity-First is sixty years old [llf]; FREESH [freesh] has already run LLF inside an LLM engine for energy-aware scheduling of a single request class. What is new here is narrower and, we argue, more useful: LLF applied *per batch job against an online tier*, carrying commercial batch-window semantics, with the same laxity arithmetic reused as the admission test.

We implement the policy at two placements and compare them. *Technique A* places all policy in a sidecar gateway: batch items are injected as low-priority requests into an unmodified vLLM, throttled by additive-increase/multiplicative-decrease control over the engine's public metrics. It tracks upstream vLLM releases with no coupling beyond the OpenAI API and `/metrics`. *Technique B* moves admission into the engine via vLLM's pluggable scheduler interface (a non-stable interface; we pin a version): a subclass wraps the stock scheduler, fills each iteration's residual token budget with batch work, bounds interference with a latency model in ConServe's context-aware form [conserve] but fitted continuously from the engine's own step timings, reserves KV-cache headroom for online bursts, and preempts the cheapest victim first. Neither placement forks vLLM.

Contributions:

1. **Batch-API contract semantics for a co-located tier**: per-job day-scale deadlines with laxity-driven escalation, a progress floor, and laxity-based admission — the customer-facing contract that co-location systems do not expose and deadline-aware schedulers do not target.
2. **A self-calibrating interference model**: technique B fits its step-latency model from live (tokens, context, time) observations with an R^2-gated fallback, removing the offline profiling sweep HyGen and ConServe require.
3. **The placement comparison**: the same policy externally vs in-engine on identical traces, revealing a throughput/latency frontier and an inversion the fork-based literature — which evaluates only its own placement — has not reported.
4. **Deployability artifacts**: durable OpenAI-wire API, crash recovery, idempotent state machines, metering, and a real client — with the operational semantics (§4) a paper usually omits and an operator cannot.

2. Background and Motivation

2.1 ONE BUDGET, TWO PRODUCTS

A vLLM V1 iteration schedules up to `max_num_batched_tokens` (τ) tokens across running and waiting requests; τ is sized so an iteration's latency fits the inter-token target. A running decode consumes one token per iteration; a prefill consumes up to a chunk. The KV cache — a separate budget — bounds how many requests may be resident. The token budget is the perishable good: unused tokens in an iteration are gone.

2.2 WHY BEST-EFFORT IS NOT A PRODUCT

Consider a large batch submitted against a deployment that then runs at sustained high online load. Best-effort co-location delivers whatever slack happens to exist; the customer learns at hour 24 how much that was. A wall-clock aging rule — "priority rises every hour, parity at hour 24" — fails twice: it escalates small on-track jobs pointlessly and escalates large at-risk jobs too late (full service beginning exactly when time has run out). Feasibility is a relation between remaining work and remaining capacity — which is laxity, and which is why the same arithmetic serves as the admission test: a job whose laxity is negative at submission should not be accepted at all.

2.3 THE PLACEMENT QUESTION

Engine schedulers act per iteration (~10–100 ms) with exact visibility of the token budget and KV state; an external controller acts at 1 Hz on aggregate metrics. Intuition says in-engine must dominate. But the engine's own priority preemption already covers token-granularity interference between controller ticks, and an external controller survives engine upgrades untouched. No prior co-location work isolates this variable; we measure it, and the intuition does not survive contact (§5).

3. Design

3.1 SYSTEM OVERVIEW

Batch jobs enter through an OpenAI-wire-compatible `/v1/files` + `/v1/batches` API; any OpenAI SDK works unchanged with a different `base_url`. Jobs are validated line-by-line, checked for admission feasibility (§3.2), and stored durably (SQLite WAL by default; the store is a repository interface, so Postgres is a DSN change). Items progress `pending → inflight → succeeded/failed` with idempotent terminal transitions. Online traffic does not traverse the gateway.

3.2 THE CONTRACT: LAXITY AT ADMISSION AND IN SCHEDULING

For each active batch b with deadline D_b, remaining items R_b, and observed completion rate r_b (sliding window; when no completion history exists the drain term is dropped — jobs escalate on evidence or deadline proximity, never on a missing denominator):

```
laxity(b)   = (D_b − now) − R_b / r_b
urgency(b)  = clamp(1 − laxity(b) / H, 0, 1)      H = escalation horizon
(6h ≪ 24h window)
priority(b) = 100 → 1 linearly with urgency        (online = 0;
sla_strict allows 0 at urgency 1)
```

Admission. At `POST /v1/batches`, the projected completion `R/r_global` is checked against the window with a configurable margin; infeasible jobs are rejected with an error naming the arithmetic. No history → accept (the system cannot judge).

Scheduling. An on-track job (laxity ≥ H) never rises above priority 100 — it exerts no *escalation* pressure whatever the clock says. (Co-location itself has a latency price even at the lowest band; §5 measures it. The escalation property is about never spending *more* online headroom than the deadline requires, not about zero cost.) The horizon, not the window, scales urgency — scaling by the window is wall-clock aging in disguise. Escalated jobs climb independently; above urgency 0.9 a token bucket guarantees a minimum admission rate; the floor's guaranteed progress feeds back into laxity, stabilizing the job at the floor. Escalation applies at submission time (vLLM fixes a request's priority at admission); the backlog, where laxity lives, is by construction the unsubmitted portion.

3.3 TECHNIQUE A: EXTERNAL CONTROL

A 1 Hz loop scrapes three public metrics, smooths them (EWMA), and adjusts a target in-flight batch count by AIMD: halve on detected online queueing (conservatively — the waiting metric is not priority-partitioned, so we assume all our in-flight items are queued) or high KV watermark; increment below a low watermark. Items are claimed in deadline order, grouped by shared prompt prefix within a batch to recover some prefix-cache locality through arrival order (we have not ablated how much of HyGen's engine-side prefix-sharing benefit [hygen] this recovers), and submitted at their batch's current laxity priority. Engine unavailability opens a circuit; in-flight items re-queue idempotently.

3.4 TECHNIQUE B: IN-ENGINE WORK CONSERVATION

`TidalScheduler` subclasses the V1 scheduler behind `--scheduler-cls` and wraps — never reimplements — the stock `schedule()`. Requests with priority ≥ 1 are the batch class (online is 0 alone; an `sla_strict` job that reaches 0 is deliberately treated as online — that *is* parity). The pre-gate hides batch waiting requests beyond a per-iteration cap X from the stock pass (restored exception-safely), where X solves $\hat{T}(P\_on + x, C) \le TBT\_budget$ with $\hat{T}(P,C) = k_1P + k_2P(P+C) + k_4(P+C) + k_5$ — ConServe's context-aware form, which prices the attention cost a pure token count misses. Coefficients fit continuously from the engine's own (P, C, step-time) stream: ring buffer, periodic least squares, non-negativity clamps, an R^2 gate with a conservative static fallback. With zero online requests resident, the cap is lifted entirely (offline mode: fill τ). A KV guardband (EWMA of online demand, clamped 5–30%) stops batch admission before online bursts can be starved of blocks; proactive eviction picks the batch victim with the least non-prefix-cached work. Enforcement is at chunk granularity — we bound

which requests the stock scheduler sees, not its inner token loop — which is what keeps the wrapper robust to upstream drift (it survived a 1,682-commit jump during development) at the cost §5 quantifies.

3.5 WHAT WE DELIBERATELY DID NOT BUILD

ConServe's layer-wise safepoint preemption and incremental KV checkpointing address unbounded iteration times; chunked prefill bounds ours to one τ-sized step. Both are natural extensions where τ-bounded iterations are still long, and §5's placement result argues sub-iteration control is exactly what technique B lacks.

4. Implementation and operational semantics

Tidal is a Python implementation over unmodified upstream vLLM (v0.26 series), with 240+ unit tests and integration tests that drive a live engine; the API layer is wire-verified against the official OpenAI SDK. The operational semantics reviewers should hold us to: **idempotency** — item claims are single atomic updates; terminal item transitions are idempotent, so a duplicate or late result never drifts `request_counts`; result-file attachment is a first-writer-wins claim, so concurrent finalization cannot split a batch's outputs. **Crash recovery** — a restarted dispatcher re-queues resultless in-flight items; re-execution rides vLLM's prefix cache. **Late results** for cancelled or expired items are recorded-then-discarded without corrupting counts, and un-wedge finalization. **Failure model** — one dispatcher per store is assumed; there is no ownership lease (a second dispatcher is safe for data, not for duplicated work — documented, with the lease as roadmap). **Metering** prices batch tokens at a configurable discount against an online price table; we deliberately make no claim about what discount a marginal-cost model would justify.

The client side reuses lazycode's provider abstraction: its Tidal adapter mirrors its Anthropic batch adapter nearly line-for-line, with the remaining friction inherited from the OpenAI Batch wire itself (no per-item expired counts; results as two files needing a union by `custom_id`; cancelled items silently absent). An engine plugin also has an observability obligation a fork does not: vLLM's logging configuration attaches handlers only to its own logger tree, and scheduler telemetry is emitted from a forked EngineCore process — technique B injects a logging configuration or runs blind.

5. Evaluation

Setup. Apple M4 Pro, CPU-only vLLM build (v0.26.1 series @ e6d67fdd), Qwen2.5-0.5B-Instruct, τ = 2048, `--scheduling-policy priority`, `--enforce-eager`. Online load: seeded Poisson arrivals of short chat requests (identical schedules across conditions); we measure total non-streaming **request latency** (not TTFT/TBT — on this configuration a request is a short prefill plus a short decode). Batch: jobs of short chat completions. **Scope disclaimer**: CPU-scale numbers characterize mechanisms and their relative behavior under one hardware regime. They support no claims about GPU efficiency, absolute capacity, or production latency; the planned GPU evaluation (8×H200) is where those claims must be earned.

Q1 — the co-location tax (1.0 req/s, 1,200-item jobs, 10-min windows). All co-location conditions completed the identical batch. The online latency price differed by an order of magnitude:

condition	online p50	online p99	vs baseline (p50 / p99)
online-only	0.44 s	1.43 s	1.00× / 1.00×
naive (priority field only)	11.97 s	32.24 s	27.3× / 22.6×
Technique A	1.12 s	2.52 s	2.54× / 1.76×
Technique B	1.26 s	11.80 s	2.86× / 8.25×

Naive co-location — what vLLM's priority field alone provides — is catastrophic despite correct priority ordering: admission priority does not bound resident-request contention. Note the medians: they degrade more than the tails everywhere (co-serving adds a near-constant per-step cost that dominates fast requests). p99-only reporting — common in this literature — flatters every technique, including ours.

Q2 — work conservation (non-draining 3,000-item pool). Offline ceiling: 4.43 items/s (68.5 output tok/s). Technique A sustains **45.1% of ceiling at 1.97× online p99** — its in-flight cap of 4 is the binding constraint, a deliberate, configurable interference ceiling. Technique B sustains **71.2% of ceiling at 9.9× online p99**. Placement is a frontier: the gateway buys latency protection with throughput; the in-engine scheduler buys

throughput with latency. For shape calibration only: HyGen and ConServe report 78–88% of ceiling on GPU-class hardware where the mechanisms that would let B have both (guardband eviction, compute-bound τ filling) actually engage.

Q3 — the contract, honestly. We attempted to demonstrate end-to-end deadline discrimination — a feasible-but-tight job that best-effort co-location misses and laxity escalation meets — and did not find a load regime on this testbed that discriminates. At 3 req/s with a 900-item/15-minute job, the batch completed with 3.5 minutes to spare even under the heaviest online load we could apply (online p99 5.8 s); at 2–2.6 req/s, every configuration completed early for both arms. On this hardware, feasible cases were easy for best-effort AIMD and we could not construct a feasible-but-tight case: **we therefore do not claim demonstrated deadline improvement.** What the runs do establish: the escalation machinery operates (per-tick priority traces show laxity-driven bands engaging), the floor engages at the configured urgency, and the admission check (§3.2) refuses jobs the observed rate cannot serve. Constructing the discriminating regime — bursty diurnal online load with genuinely scarce slack — is a primary goal of the GPU evaluation.

Q4 — placement, and an inversion. The naive expectation — in-engine must dominate — is wrong on this hardware. Technique B's cap and guardband address interference channels that never bind here: peak KV usage in every run was below 0.5%, so guardband eviction never fired, and a resident batch request decodes every iteration regardless of the admission cap. The dominant interference channel on this box is resident-decode contention, which only sub-iteration participation control could bound (§7). Technique A holds latency (1.76× vs 8.25× at equal work; 1.97× vs 9.9× at capacity) because its control point — how many batch requests exist at all — is upstream of residency; B converts that residency into 26 more points of ceiling instead. We expect the frontier to shift, and possibly invert back, on hardware where compute rather than residency binds; measuring that crossover is the placement question's real test. To our knowledge no prior work reports this comparison; the fork-based literature evaluates only its own placement.

Q5 — self-calibration. From cold start, $\hat{T}$ passed its R^2 gate within the first capped phase of the integration run and settled at MAPE ≈ 0.11 on live traffic; the batch-token cap X moved from the cold-start 40 to a converged 1 — correctly tight, since at a 60 ms budget with ~25 ms online-only steps this box has little slack to sell, and the model discovered that. Regime-change adaptation (model swap, thermal drift) is untested (§7).

6. Related Work

Iteration-level scheduling and its descendants frame the mechanism space: Orca introduced iteration-level batching [orca]; Sarathi-Serve added chunked prefill and stall-free batching, the token-budget structure vLLM V1 inherits [sarathi]; FastServe [fastserve] brought preemptive MLFQ scheduling to inference; Llumnix [llumnix] reschedules across instances by live migration; DistServe and Splitwise disaggregate prefill from decode [distserve, splitwise], an architecture ConServe found interacts poorly with offline co-location; InferCept's preemption taxonomy [infercept] underlies HyGen's KV-handling choices. Co-location: HyGen [hygen] (profiled latency budget, prefix-sharing offline order, starvation tempering — but no per-job deadline or admission contract), ConServe [conserve] (context-aware latency model, layer-wise preemption, incremental KV checkpointing — fork, no contract), OOCO [ooco], Echo [echo]. Deadline-aware serving: QLM [qlm] (ILP queue reordering; evaluates EDF as a baseline it beats), Ascendra [ascendra], SCORPIO [scorpio], and closest, Niyama [niyama] — per-request deadlines across QoS classes in one engine, with EDF/SRPF-interpolated priority at second-to-minute horizons and relegation under overload; it does not expose per-job day-scale windows, admission refusal, or laxity. FREESH [freesh] implements true LLF in an LLM engine for carbon-aware frequency scaling over one request class — evidence the math runs in this setting, aimed at a different problem. Within vLLM: priority scheduling was motivated by batch/interactive co-location [vllm-rfc-6077], an SLA-tier proposal was declined [vllm-rfc-30256], and an online batch-API endpoint remains an open PR [vllm-pr-44445] — demand without an assembled answer. Every individual mechanism here is published; the contribution is the contract semantics, the synthesis, and the placement measurement.

7. Limitations

Admission gating does not bound resident-request inflation (technique B): a resident batch request keeps decoding until the guardband evicts it, and where KV pressure never materializes, eviction never fires — the dominant residual interference on our testbed. Bounding it requires per-iteration participation control (abandoning the wrapper's upstream tolerance) or ConServe-style sub-iteration preemption. **The contract is mechanism-validated, not outcome-demonstrated**: no tested regime discriminated escalation end-to-end (§5 Q3), and the `sla_strict` dial's latency price is unmeasured. **Tail metrics flatter co-location**; we report distributions. **CPU testbed**: no

GPU claims are supported. **Median tax**: even technique A costs ~2.5× p50 on this box; co-location is not free and we do not claim it is. Single engine, single model; one dispatcher per store (no lease); submission-time escalation cannot re-age already-queued items; step-time attribution is exact on the CPU build and approximate under GPU async scheduling; `scheduler_cls` is not a stable interface (technique B pins a version; technique A does not).

8. Conclusion

The batch API is the rare product where the entire margin is scheduling. Tidal implements its contract — admission, escalation, floor — over an unmodified open-source engine with classical real-time machinery, measures what each placement of that policy costs, and is explicit about what a modest testbed can and cannot demonstrate. We offer it as usable infrastructure, as a baseline for deadline-contracted serving, and as a placement study the fork-based literature has not run — with the GPU-scale evaluation as the necessary next step.

References

[sarathi] A. Agrawal et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024. arXiv:2403.02310. [orca] G.-I. Yu et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022. [fastserve] B. Wu et al. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920. [llumnix] B. Sun et al. Llumnix: Dynamic Scheduling for Large Language Model Serving. OSDI 2024. arXiv:2406.03243. [distserve] Y. Zhong et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving. OSDI 2024. arXiv:2401.09670. [splitwise] P. Patel et al. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. ISCA 2024. arXiv:2311.18677. [hygen] T. Sun, P. Wang, F. Lai. HyGen: Efficient LLM Serving via Elastic Online-Offline Request Co-location. arXiv:2501.14808. [conserve] Y. Qiao et al. ConServe: Fine-Grained GPU Harvesting for LLM Online and Offline Co-Serving. arXiv:2410.01228. [niyama] Niyama: Breaking the Silos of LLM Inference Serving. arXiv:2503.22562. [freesh] FREESH: Fair, Real-time, Energy-Efficient Scheduling for LLM Serving. arXiv:2511.00807. [qlm] QLM: Queue Management for SLO-Oriented Large Language Model Serving. SoCC 2024. arXiv:2407.00047. [ascendra] Ascendra: Dynamic Request Prioritization for Efficient

LLM Serving. arXiv:2504.20828. [scorpio] SCORPIO: Serving the Right Requests at the Right Time. arXiv:2505.23022. [ooco] Online-Offline Co-location for LLM Serving. arXiv:2511.21862. [echo] Echo: Efficient Co-scheduling of Hybrid Online-Offline Tasks. arXiv:2504.03651. [llf] A. K. Mok. Fundamental Design Problems of Distributed Systems for the Hard-Real-Time Environment. MIT, 1983. [borg] A. Verma et al. Large-scale cluster management at Google with Borg. EuroSys 2015. [vllm] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180. [infercept] R. Abhyankar et al. InferCept: Efficient Intercept Support for Augmented LLM Inference. ICML 2024. arXiv:2402.01869. [vllm-rfc-6077] vLLM RFC: Priority Scheduling. github.com/vllm-project/vllm/issues/6077. [vllm-rfc-30256] vLLM RFC: SLA-Tiered Scheduling (closed, not planned). github.com/vllm-project/vllm/issues/30256. [vllm-pr-44445] vLLM PR: OpenAI-compatible online Batch and Files API (open). github.com/vllm-project/vllm/pull/44445. [lazycode] lazycode: plan with a realtime LLM, execute on batch APIs. github.com/rajagurunath/lazycode.